Lecture 08 — The for loop, in practice — and the project begins

Algorithmic Thinking · Week 4, Session 8 — Friday 16 October 2026, 17:35–18:55 (after Lab 4, 16:30–17:30) · Situación problema formally released

Leopoldo Rodríguez Salazar · Tecnológico de Monterrey

Contents

Overview 1
Opener — Which book do you throw out? (10 min) 1
Notes 2
Situación problema — formally released (12 min) 4
Stretch — if the session was easy 4
After the session 5

Objective. Write for loops that read, choose and draw — including a loop inside a loop — and leave with a project partner and a dataset to look for.

Overview

Tuesday gave you `for`. Today you write four of them, one inside another, and then we stop being a course about loops for fifteen minutes and become a course about **your** data: the Situación problema is formally released, teams are formed, and the first deliverable has a date.

The four exercises are the four things `for` is for: **read exactly n things**, **keep the best one**, **do something for every pair** (a loop inside a loop), and **draw**. None is hard. All four have a boundary — `n = 0`, `n = 1`, the first row — and the shell is where you check it.

Opener — Which book do you throw out? (10 min)

Algorithms to Live By, chapter 4, *Caching*. Your backpack holds four books. You are carrying four, and you need a fifth. One has to go. Which?

Take three answers. “The biggest.” “The one I like least.” “The one I used longest ago.” The third one is the computer’s answer, and it has a name: **LRU — Least Recently Used**. Every cache in every computer you own — the processor’s, the browser’s, your phone’s — evicts the item that was touched longest ago, because the best predictor of *needing something soon* is *having needed it recently*. Belady proved in 1966 that the truly optimal rule needs to know the future; LRU is the best you can do without it. The book’s extension: your desk, your closet, your phone’s home screen — put what you used last on top, and the thing at the bottom is the one to drop.

How do you *find* the least recently used of four books? You look at each one, remember the oldest so far, and replace it when you see an older one. That is a loop over a known number of things with a **best-so-far** inside it — today's exercise 2, with "oldest" instead of "largest".

Your turn, 3 minutes, on paper: `INPUT:` four books, each with the date you last opened it. `OUTPUT:` the one to remove. Numbered steps: the loop, the memory for the oldest so far, the comparison. **Underline the line that says how many times the loop runs.**

Notes

What broke in the lab?

Two answers. You want "the menu kept repeating after *Salir*" (the flag `continua` never set to `False`) or "`factores_de(12)` printed 12" (the `range` end). Both are today's `+ 1` and flag, from Tuesday, meeting real code.

Exercise 1 — `average_of(n)` *(read exactly n things)*

Open `01_average_of.py`:

```
def average_of(n):
    total = 0
    for i in range(n):
        prompt = "Number " + str(i + 1) + ": "
        value = float(input(prompt))
        total = total + value
    return total / n
```

Shell: `average_of(3)` with `10 20 30` → `20.0`. The `for` counts the inputs; no `asked` variable, no step line — compare with lecture 5's `positive_sum`. The prompt shows `i + 1` because `range(3)` gives 0, 1, 2 and humans count from 1; `str(i + 1)` turns the number into text so `+` can join it.

Now the boundary: `average_of(0)`. The loop runs zero times — no error — and then `0 / 0`. The margin note from week 1, for the last time: the loop is safe, the line *after* it is not, so `main` refuses `n <= 0` before calling.

Exercise 2 — `max_of(n)` *(keep the best one)*

Open `02_max_of.py`:

```
def max_of(n):
    best = float(input("Number 1: "))
    for i in range(2, n + 1):
        prompt = "Number " + str(i) + ": "
        value = float(input(prompt))
        if value > best:
            best = value
    return best
```

Shell: `max_of(3)` with `4 9 2` → `9.0`. `max_of(1)` with `7` → `7.0`: the loop runs zero times and the first value *is* the answer. That is the opener's LRU loop with `>` instead of "older

than". Compare with lecture 5's `maximum = None`: there we did not know whether any value would come; here `n ≥ 1` is promised, so the first read *is* the start. **The start of a best-so-far is the first candidate, not an invented one.** (`best = 0` would be wrong for `-4 -9 -2`.)

Exercise 3 — `table(n)` and `all_tables(up_to)` *(a loop inside a loop)*

Open `03_table.py`:

```
def table(n):
    for i in range(1, 11):
        print(n, "x", i, "=", n * i)

def all_tables(up_to):
    for n in range(1, up_to + 1):
        table(n)
        print()
```

`table(7)` prints ten lines. `all_tables(3)` prints the tables of 1, 2 and 3 — **thirty** lines plus three blank ones. Before running `all_tables(12)`, say how many lines: 12 × 10 = 120. The inner loop runs completely for every single turn of the outer one; turns multiply. When a program is slow, this is where to look first — a loop inside a loop inside a loop is n^3, and `n = 1000` is a billion.

Notice the shape: the inner loop is *a function call*. `all_tables` does not know how `table` works and does not need to. That is week 2's lesson applied to loops — and it is how your project's `main()` will call `grafica1(tablero)` without knowing what a plot is.

Exercise 4 — `bars(n)` and `pyramid(n)` *(draw)*

Open `04_bars.py`:

```
def bars(n):
    for i in range(1, n + 1):
        print("*" * i)

def pyramid(n):
    for i in range(1, n + 1):
        print(" " * (n - i) + "*" * (2 * i - 1))
```

`bars(3)` → *, **, ***. `"*" * i` is a string repeated `i` times — the thing QE1's A2 warned you about (`"5" + "3"` is `"53"`), now useful. `pyramid(3)` needs `n - i` spaces then `2i - 1` stars; trace row 1 before running: 2 spaces, 1 star. `bars(0)` prints nothing and raises nothing — `range(1, 1)` is empty.

Exercise 5 — `spread(trials, repeats)` *(Monte Carlo's price)*

Open `05_spread.py` — Tuesday's stretch 1, as a program: run `estimate_pi(trials)` ten times, report largest minus smallest, for 100, 10 000 and 1 000 000 darts. Typical: `0.7`, `0.05`, `0.005`. **A hundred times more work buys ten times less spread.** The square root. That is why Monte Carlo is a last resort and why, when it is the only resort, it needs

a fast computer. (Your laptop did a million darts in about a second. In 1946 that was the entire budget.)

In class — the four, as files *(individual, from 18:20; what is not finished is yours to finish, evidence box only)*

File	Function(s)	Shell tests to write down
`A_Matricula_x1.py`	`average_of(n)`; `main` refuses `n <= 0` with `Need at least one number.`	`average_of(3)` with 10 20 30 → 20.0; `main` with 0 → the message
`A_Matricula_x2.py`	`max_of(n)`, same `main` rule	`max_of(3)` with 4 9 2 → 9.0; `max_of(1)` with 7 → 7.0; `max_of(3)` with −4 −9 −2 → −2.0
`A_Matricula_x3.py`	`table(n)`, `all_tables(up_to)`	`all_tables(3)` — count the lines before running: 30
`A_Matricula_x4.py`	`bars(n)`, `pyramid(n)`	`bars(0)` → nothing; `pyramid(3)` → the three rows, exactly

Algorithm on paper first for 1 and 2 (the other two are short enough to write the `FOR` line and the range in the comment). Photos + files → **Week 4 evidence box**. Finished? Make `pyramid` hollow.

Situación problema — formally released (12 min)

Open `situacion-problema.html` on the projector. Everything there has been public for three weeks; today it becomes real:

1. **Teams of two, today.** Names to me before you leave — a photo of the list on the board is fine. No team of one; a team of three only if the room is odd, and I pick it.
2. **Deliverable 1 — the dataset — Friday 6 November.** 30–50 rows, up to 10 columns, half text half numbers, from a real source (Kaggle, datos.gob.mx, the World Bank, a business that lets you). **Start looking this week.** The best projects come from data you already care about.
3. **The rubric is published** — on the site, under *How it is graded*. Each deliverable is graded on its own; the final weighs the program, the report and a two-minute oral explanation of *your* code. A program you cannot explain is not yours.
4. **The skeleton you already wrote:** this afternoon's lab — a `menu()`, a `main()` with a `while` that repeats until *Salir*, `if`/`elif` calling functions. That is the final program's `main()`, verbatim. The functions it will call — statistics, two charts — are weeks 7 to 9.

Stretch — if the session was easy

Optional, ungraded, harder than they look.

1. **Belady's ghost.** LRU is the best rule that does not know the future. Invent a sequence of book requests, for a backpack of two, where LRU evicts the wrong book every single time — and write the rule that, *knowing the sequence*, would never miss. (That rule is Belady's; it cannot be run, only admired.)
2. **The table, transposed.** Print the 10 × 10 multiplication table as a grid — ten numbers per row, aligned — using `print(..., end=" ")` to stay on the line and `print()` to end it. Then make the columns line up for every width (hint: `str(n * i).rjust(4)`).
3. **Sieve.** `count_primes(limit)` from Tuesday calls `is_prime` for every number — a loop inside a loop, `n²`-ish. Eratosthenes (240 BC) did it with one list and no division. Read about the sieve, write its algorithm on paper (you cannot code it until week 7 — no lists yet), and say why it is faster in one sentence about nested loops.

After the session

Homework 5 — `assignments/hw-05-for.md`. Due Sunday 18 October, 23:59, on Canvas. `is_perfect(n)` and `longest_collatz(limit)` — a `for` around your Homework 4 function — plus a `main`. One sentence per `for` saying how many times it runs.

Also before Tuesday:

1. **Lab 4 file** — `D_matricula.py` + the photo of your algorithm → Canvas → *Laboratory 4*, **Sunday 18 October, 23:59**.
2. **Quick exam 4 — Tuesday 20 October, first 20 minutes:** the `for` loop. Then **the midterm, Friday 23 October**, in class: weeks 1 to 4, the academia's format, 20 minutes of questions plus a program. The *Self-study: prepare for your midterm* page in Canvas is the map.
3. **Your team and your data.** Names today; a dataset candidate by Tuesday is early — and early is the whole game with data.